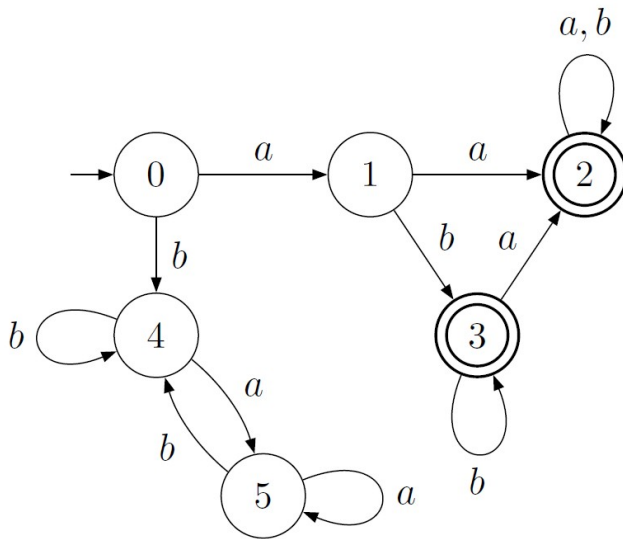




G52LAC (COMP2040)

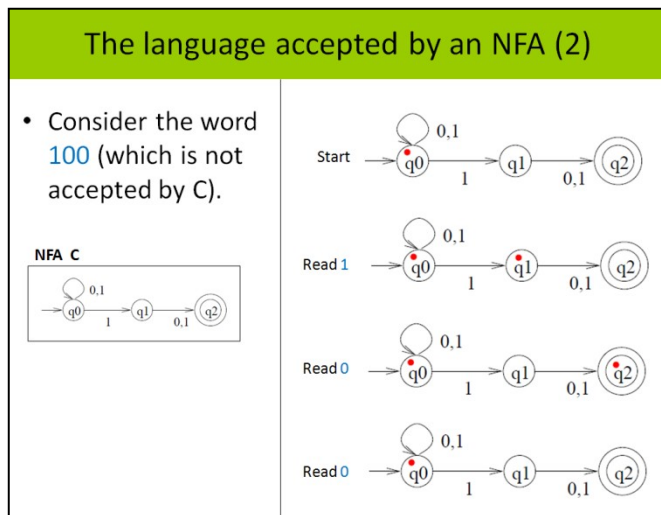
Languages and Computation

Lecture 4: DFA/NFA equivalence and other matters

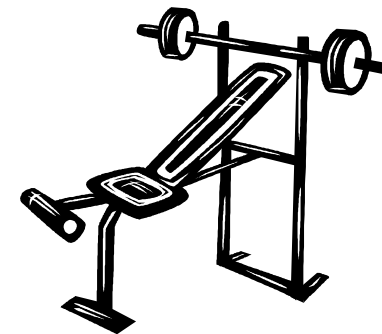
Tomas Maul
School of Computer Science
University of Nottingham Malaysia Campus

Outline

- Quick recap.
- Equivalence between NFAs and DFAs
- Lazy Subset Construction
- Proof
- Lexical Analysis



+



DFA/NFA exercises

Outline

- Quick recap.
- Equivalence between NFAs and DFAs
- Lazy Subset Construction
- Proof
- Lexical Analysis

DFA's are NFA's

- A DFA is just a special case of an NFA, where there is:
 - exactly one initial state;
 - exactly one transition from each state per symbol.
- Thus all DFA transition diagrams also define an NFA that accepts the same language.

Converting DFAs to NFAs

Given a DFA

$$A = (Q, \Sigma, \delta_A, q_0, F)$$

an equivalent NFA $N(A)$ can be constructed as follows:

$$N(A) = (Q, \Sigma, \delta_{N(A)}, \{q_0\}, F)$$

where

$$\delta_{N(A)}(q, x) = \{\delta_A(q, x)\}$$

NFA observations

- An NFA is always in a **set** of states.
- When reading an input symbol, the machine enters a new set of states.
- How many possible sets of states are there?
 - For each state, the machine is either in that state or not — i.e. 2 possibilities per state.
 - Thus $2^{|Q|}$ possibilities.
- This could be a lot, but it is **finite**.
- So we could convert an NFA to a DFA by taking each **set** of NFA states to be a **single** DFA state!

A **DFA** computes by going from **one state** to **another**.
An **NFA** computes by going from **one configuration of states** to **another**.

Converting NFAs to DFAs: The Subset Construction

Given an NFA

You must see P interchangeably as a DFA state and a corresponding set of NFA states.

$$A = (Q, \Sigma, \delta_A, S, F_A)$$

an equivalent DFA $D(A)$ can be constructed as follows:

$$D(A) = (\mathcal{P}(Q), \Sigma, \delta_{D(A)}, S, F_{D(A)})$$

where

$$\delta_{D(A)}(P, x) = \bigcup \{ \delta_A(q, x) \mid q \in P \}$$

$$F_{D(A)} = \{ P \mid P \in \mathcal{P}(Q) \wedge (P \cap F_A \neq \emptyset) \}$$

Converting NFAs to DFAs: The Subset Construction

Page 12 example

set that contains at least a final state of the NFA. The transitions just follow the active set of markers, i.e. a state $P \in \mathcal{P}(Q)$ corresponds to having markers on all $q \in S$ and when we follow the arrow labelled x we get the set of states that are marked after reading x .

As an example let us consider the NFA C above. We construct a DFA $D(C)$

$$D(C) = (\mathcal{P}(\{q_0, q_1, q_2\}), \{0, 1\}, \delta_{D(C)}, \{q_0\}, F_{D(C)})$$

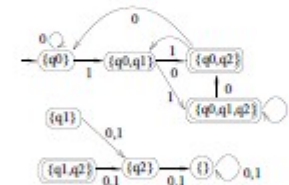
with $\delta_{D(C)}$ given by

$\delta_{D(C)}$	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1\}$	$\{q_2\}$	$\{q_2\}$
$*\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$
$*\{q_0, q_2\}$	$\{q_0\}$	$\{q_0, q_1\}$
$*\{q_1, q_2\}$	$\{q_2\}$	$\{q_2\}$
$*\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$

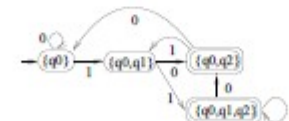
and $F_{D(C)}$ is the set of all the states marked with $*$ above, i.e.

$$F_{D(C)} = \{\{q_2\}, \{q_0, q_2\}, \{q_1, q_2\}, \{q_0, q_1, q_2\}\}$$

Looking at the transition diagram:



we note that some of the states ($\emptyset, \{q_1\}, \{q_2\}, \{q_1, q_2\}$) cannot be reached from the initial state, which means that they can be omitted without changing the language. Hence we obtain the following automaton:

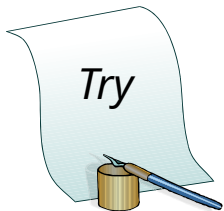
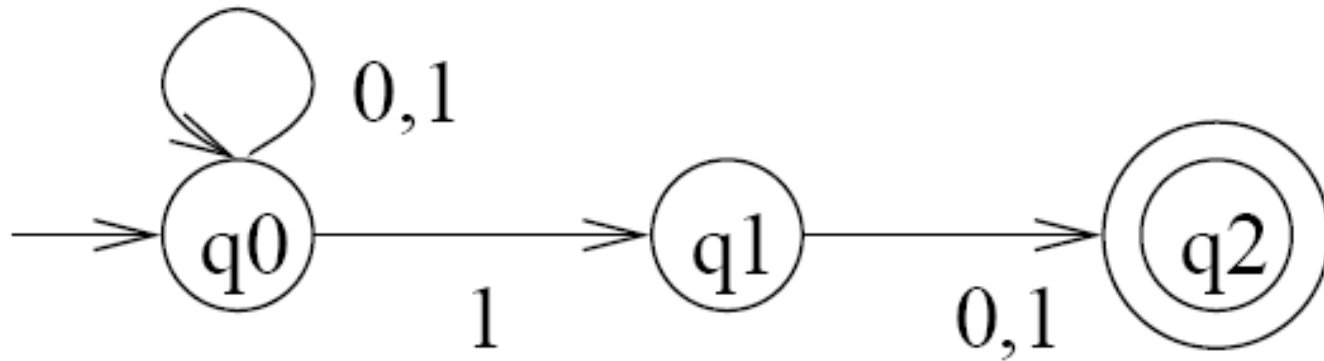


We still have to convince ourselves that the DFA $D(A)$ accepts the same language as the NFA A , i.e. we have to show that $L(A) = L(D(A))$.

As a lemma we show that the extended transition functions coincide:

Converting NFAs to DFAs: The Subset Construction

As an example let us consider the NFA C above.

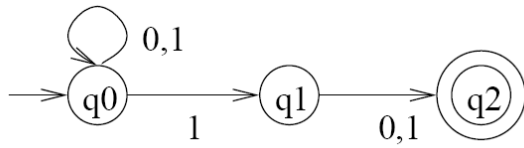


Try to construct the transition table for the corresponding DFA.

Converting NFAs to DFAs: The Subset Construction

$$D(C) = (\mathcal{P}(\{q_0, q_1, q_2\}, \{0, 1\}, \delta_{D(C)}, \{q_0\}, F_{D(C)})$$

with $\delta_{D(C)}$ given by

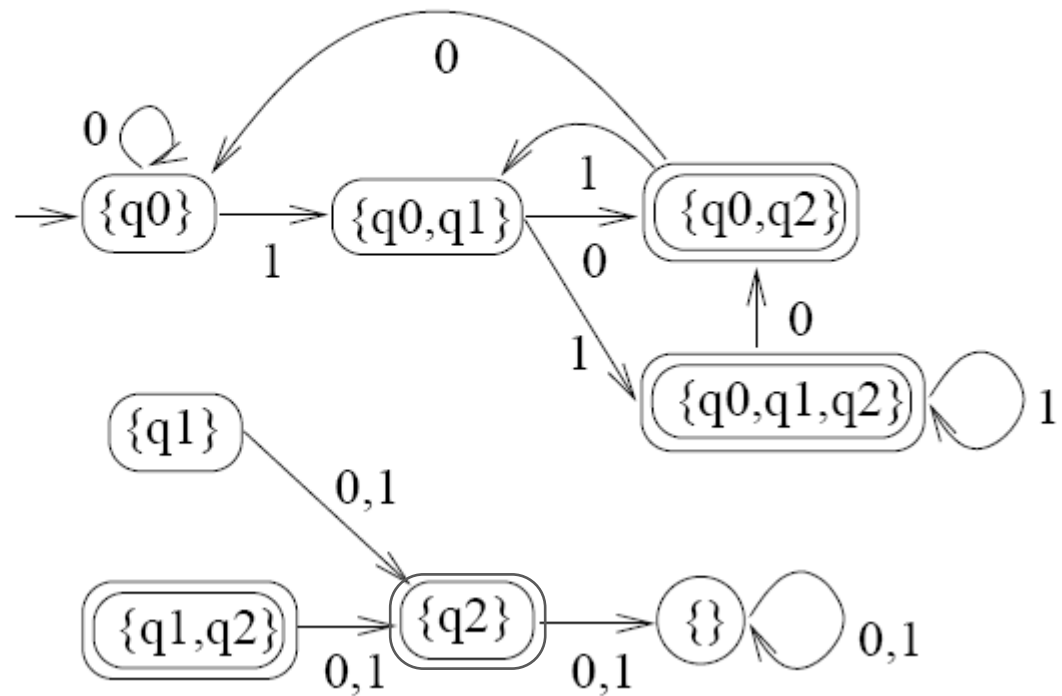


$\delta_{D(C)}$	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1\}$	$\{q_2\}$	$\{q_2\}$
$*\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$
$*\{q_0, q_2\}$	$\{q_0\}$	$\{q_0, q_1\}$
$*\{q_1, q_2\}$	$\{q_2\}$	$\{q_2\}$
$*\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$

Converting NFAs to DFAs: The Subset Construction

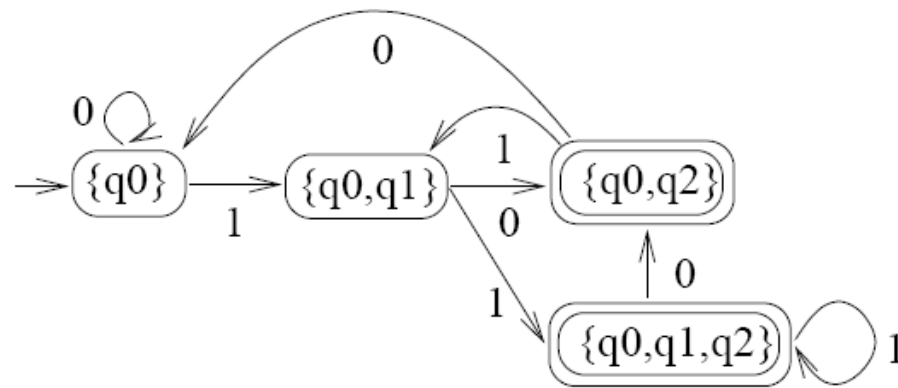
Looking at the transition diagram:

$\delta_{D(C)}$	0	1
$\emptyset$	$\emptyset$	$\emptyset$
$\rightarrow \{q_0\}$	$\{q_0\}$	$\{q_0, q_1\}$
$\{q_1\}$	$\{q_2\}$	$\{q_2\}$
$*\{q_2\}$	$\emptyset$	$\emptyset$
$\{q_0, q_1\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$
$*\{q_0, q_2\}$	$\{q_0\}$	$\{q_0, q_1\}$
$*\{q_1, q_2\}$	$\{q_2\}$	$\{q_2\}$
$*\{q_0, q_1, q_2\}$	$\{q_0, q_2\}$	$\{q_0, q_1, q_2\}$



Converting NFAs to DFAs: The Subset Construction

we note that some of the states ($\emptyset$, $\{q_1\}$, $\{q_2\}$, $\{q_1, q_2\}$) cannot be reached from the initial state, which means that they can be omitted without changing the language. Hence we obtain the following automaton:



Outline

- Quick recap.
- Equivalence between NFAs and DFAs
- Lazy Subset Construction
- Proof
- Lexical Analysis

Recap: Subset Construction

Given an NFA

$$A = (Q, \Sigma, \delta_A, S, F_A)$$

an equivalent DFA (one that accepts the same language) can be constructed as follows:

$$D(A) = (\mathcal{P}(Q), \Sigma, \delta_{D(A)}, S, F_{D(A)})$$

where

$$\delta_{D(A)}(P, x) = \bigcup \{ \delta_A(q, x) \mid q \in P \}$$

$$F_{D(A)} = \{ P \mid P \in \mathcal{P}(Q) \wedge (P \cap F_A \neq \emptyset) \}$$

Avoiding Work

- Often, many states in the equivalent DFA are **unreachable** from the initial state.
- We could avoid including those unreachable states, and thus save a lot of work.

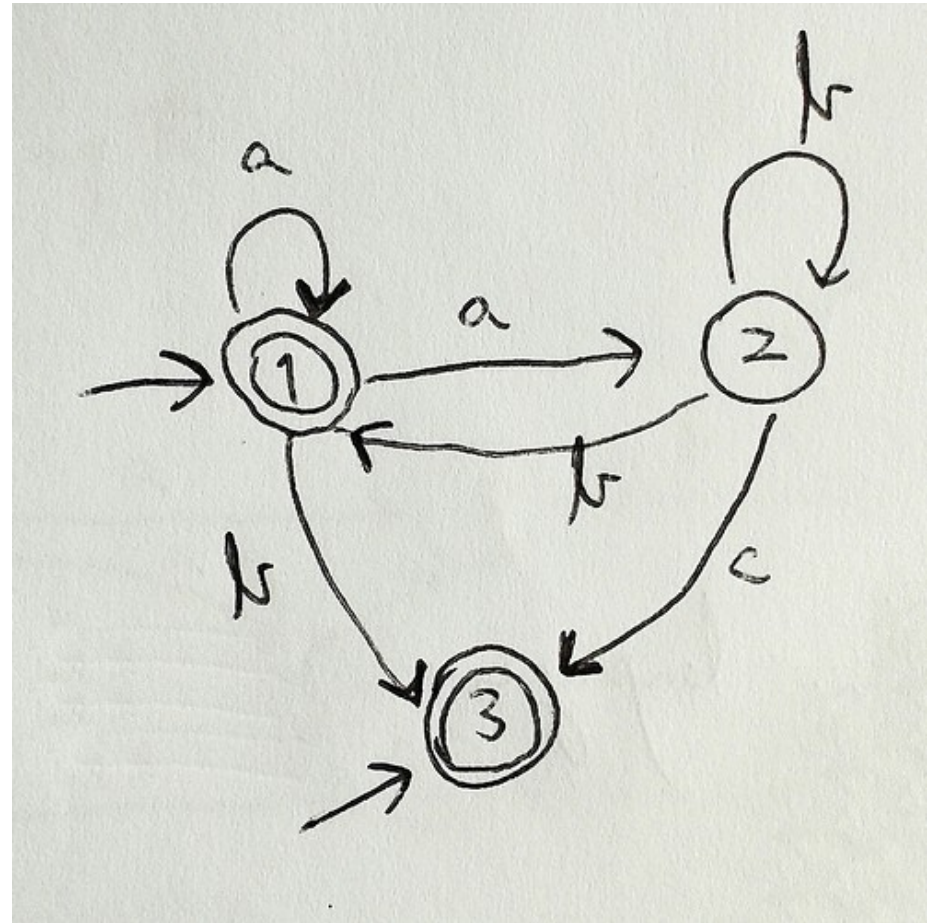
Lazy subset construction method

- Start with the DFA state corresponding to the NFA's set of start states.
- Do the first set of transitions and continue doing transitions as long as new DFA states get created.
- For every new DFA state created compute the corresponding transitions.

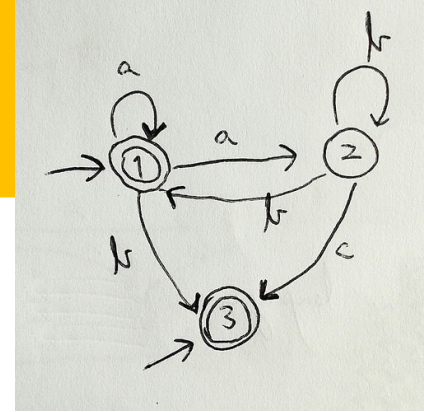
Example



1. Convert the following NFA into a DFA.
2. Start with the transition table.
3. You can use the lazy subset construction method.



Example



Transition table:

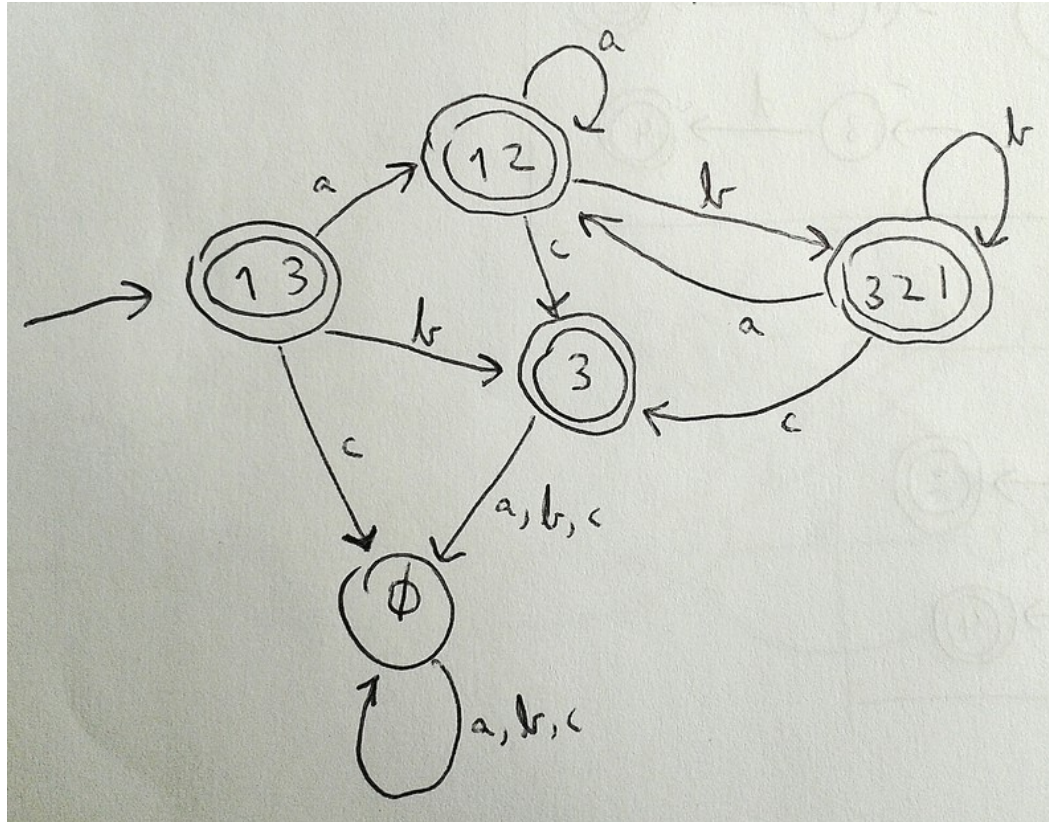
	a	b	c
→ * {1, 3}	{1, 2}	{3}	∅
* {1, 2}	{1, 2}	{3, 2, 1}	{3}
* {3}	∅	∅	∅
∅	∅	∅	∅
* {3, 2, 1}	{1, 2}	{3, 2, 1}	{3}

Example



	a	b	c
→ *{1,3}	{1,2}	{3}	$\emptyset$
*{1,2}	{1,2}	{3,2,1}	{3}
*{3}	$\emptyset$	$\emptyset$	$\emptyset$
$\emptyset$	$\emptyset$	$\emptyset$	$\emptyset$
*{3,2,1}	{1,2}	{3,2,1}	{3}

Transition diagram:

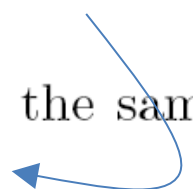


Outline

- Quick recap.
- Equivalence between NFAs and DFAs
- Lazy Subset Construction
- Proof
- Lexical Analysis

Converting NFAs to DFAs: The Subset Construction

Proof needed ...

We still have to convince ourselves that the DFA $D(A)$ accepts the same language as the NFA A , i.e. we have to show that $L(A) = L(D(A))$. 

As a lemma we show that the extended transition functions coincide:

Lemma 2.1

$$\hat{\delta}_{D(A)}(P, w) = \hat{\delta}_A(P, w)$$

The result of both functions are sets of states of the NFA A : for the right hand side because the extended transition function on NFAs returns sets of states and for the **left** hand side because the states of $D(A)$ are sets of states of A .

Recap induction ...

The way the natural numbers are generated helps us not only in defining functions, but also in proving properties. If we want to prove that a certain statement about numbers is true for every element of $\mathbb{N}$, we can do it by showing that it is true for 0 and that if it is true for a number k , then it must be true for the next number $k + 1$.

This method to prove propositions about natural numbers is called *induction*. Suppose that the property you're interested in is called P . To say that a number n satisfies this property, we write $P(n)$.

Induction Principle: if $P(0)$
 and $P(k) \Rightarrow P(k + 1)$ for all numbers k
 $\frac{\text{then } P(n) \text{ for all natural numbers } n.}{\text{then } P(n) \text{ for all natural numbers } n.}$

The idea is that we can prove that P holds for all natural numbers one step at a time:

(see next slide)

- $P(0)$ holds by hypothesis;
- Instantiating the step case with $k = 0$ we get that $P(0) \Rightarrow P(1)$ and so, since we already know that $P(0)$ is true from the previous step, also $P(1)$ is true. (This is the rule of implication elimination, if you remember well.)
- Instantiating the step case with $k = 1$ we get that $P(1) \Rightarrow P(2)$ and so, since we already know that $P(1)$ is true from the previous step, also $P(2)$ is true.
- With $k = 2$ we get that $P(2) \Rightarrow P(3)$, and $P(2)$ is true by the previous step, so also $P(3)$ is true.
- With $k = 3$ we get that $P(3) \Rightarrow P(4)$, and $P(3)$ is true by the previous step, so also $P(4)$ is true.
- And so on for all the natural numbers.

The two cases of the proof by induction are called the **base case** ($P(0)$) and the **induction case** ($P(k) \Rightarrow P(k+1)$). When proving the induction case, we make the assumption $P(k)$, which is called the **induction hypothesis**, and from it we must prove the conclusion $P(k+1)$.

A note on the base case.

In some contexts, where **zero** is considered the first natural number (e.g. combinatorics and mathematical logic), the base case is **P(0)**, whereas in other contexts, where the number **one** is considered the first natural number, the base case is **P(1)**.

Converting NFAs to DFAs: The Subset Construction

$$\hat{\delta}_{D(A)}(P, w) = \hat{\delta}_A(P, w)$$

Proof: We show this by *induction over the length of the word w* , let's write $|w|$ for the length of a word.

$|w| = 0$ Then $w = \epsilon$ and we have

$$\begin{aligned} \hat{\delta}_{D(A)}(P, \epsilon) &= P && \text{by (1)} \\ &= \hat{\delta}_A(P, \epsilon) && \text{by (3)} \end{aligned}$$

$$\hat{\delta}(q, \epsilon) = q$$

$$\hat{\delta}(P, \epsilon) = P$$

$|w| = n + 1$ Then $w = xv$ with $|v| = n$.

$$\begin{aligned} \hat{\delta}_{D(A)}(P, xv) &= \hat{\delta}_{D(A)}(\delta_{D(A)}(P, x), v) \\ &= \hat{\delta}_A(\delta_{D(A)}(P, x), v) \\ &= \hat{\delta}_A(\bigcup \{\delta_A(q, x) \mid q \in P\}, v) \\ &= \hat{\delta}_A(P, xv) \end{aligned}$$

by (2)

ind.hyp.

by construction

by (4)

subset construction method

$$\hat{\delta}(q, xw) = \hat{\delta}(\delta(q, x), w)$$

□

$$\hat{\delta}(P, xw) = \hat{\delta}(\bigcup \{\delta(q, x) \mid q \in P\}, w)$$

Converting NFAs to DFAs: The Subset Construction

Theorem 2.2

$$L(A) = L(D(A))$$

Proof:

$$\begin{aligned} & w \in L(A) \\ \iff & \text{Definition of } L(A) \text{ for NFAs} \\ & \hat{\delta}_A(S, w) \cap F \neq \emptyset \\ \iff & \text{Lemma 2.1} \longrightarrow \hat{\delta}_{D(A)}(P, w) = \hat{\delta}_A(P, w) \\ & \hat{\delta}_{D(A)}(S, w) \cap F \neq \emptyset \\ \iff & \text{Definition of } F_{D(A)} \\ & \hat{\delta}_{D(A)}(S, w) \in F_{D(A)} \longrightarrow \text{Refer to slide 7} \\ \iff & \text{Definition of } L(A) \text{ for DFAs} \\ & w \in L_{D(A)} \end{aligned}$$

□

Corollary 2.3 *NFAs and DFAs recognize the same class of languages.*

Proof: We have noticed that DFAs are just a special case of NFAs. On the other hand the subset construction introduced above shows that for every NFA we can find a DFA that recognizes the same language. □

Summary

- A DFA is a special case of an NFA.
- An NFA can be converted to a DFA using the Subset Construction.
- Thus DFAs and NFAs are **interconvertible**, and therefore equivalent in the sense that they characterise exactly the same class of languages: the **Regular Languages**.

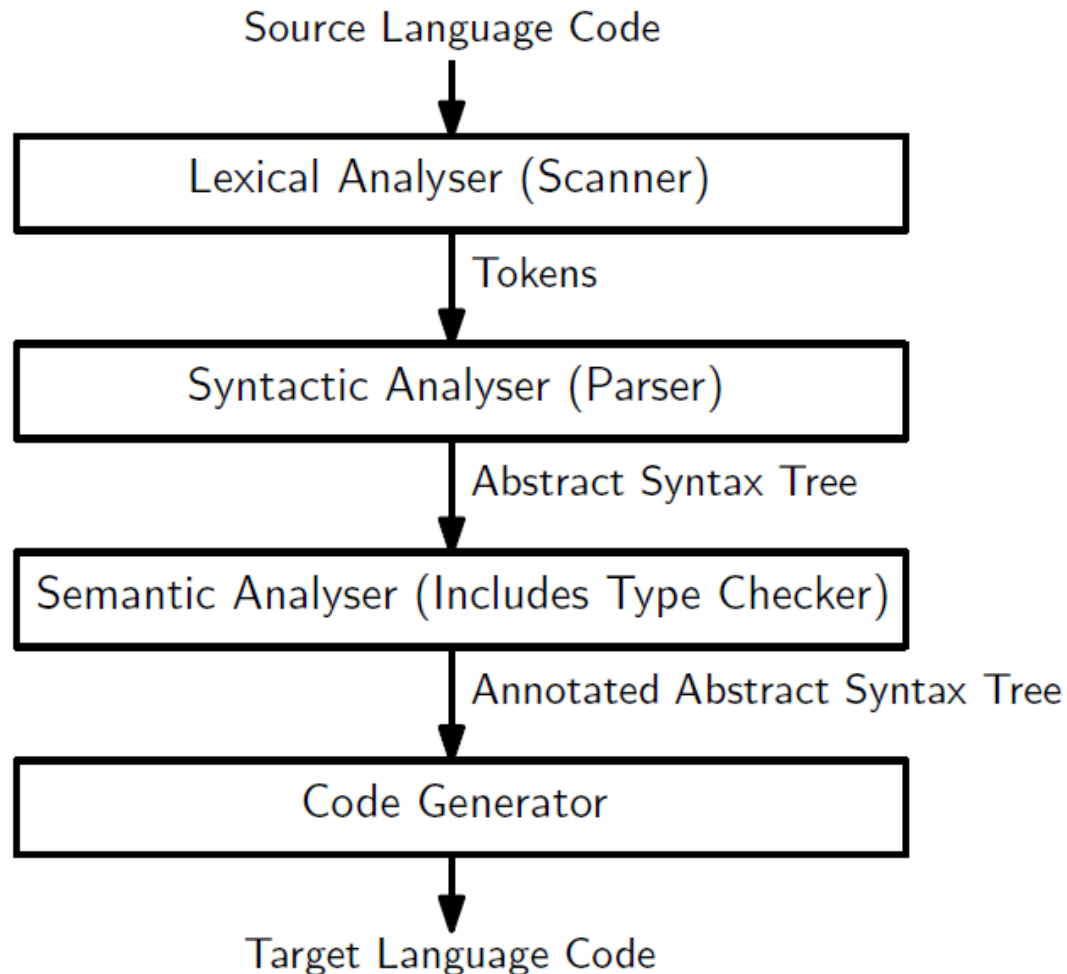
Outline

- Quick recap.
- Equivalence between NFAs and DFAs
- Lazy Subset Construction
- Proof
- Lexical Analysis

Lexical Analysis

- First phase of a compiler
- Common application of finite automata
- Also known as **scanning** or **lexing**

Compiler Structure

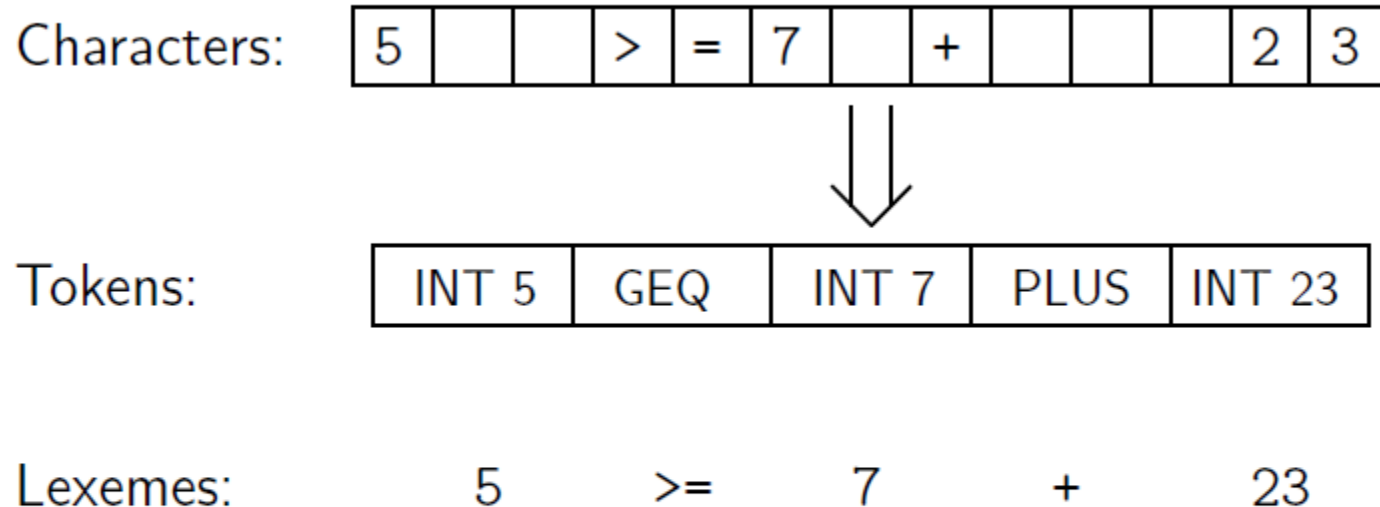


Terminology



- Characters: the alphabet of the source program (often ASCII)
- Token: a name and (optionally) an associated value
- Lexeme: a sequence of characters that correspond to a token

Example



Note that if > and = are also lexemes, care must be taken to ensure that two tokens are not produced for >=

Finite Automata

- How do finite automata fit into this?
- They can be used to recognize lexemes.
- Each accepting state is annotated with a token to produce.

Further Reading

- Compilers: Principles, Techniques, & Tools (2nd Edition), pages 109–120 & 130–134

Reading

- Introduction to Automata Theory, Languages, and Computation (3rd edition), pages 60–71
- G52MAL Lecture Notes, pages 11–13

Acknowledgements

- Thorsten Altenkirch.
- Neil Sculthorpe.
- Henrik Nilsson.
- V.P. Kallimani.
- Venanzio Capretta.